

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

По лабораторной работе №8

По дисциплине «Логика и основы алгоритмизации в инженерных
задачах»

На тему «Обход графа в ширину»

**Выполнил студент группы
24ВВВ2:**

Тремасов К. И.

Приняли:

Юрова О.В.

Деев М.В.

Пенза, 2025

Цель работы

Научиться реализовывать алгоритм для обхода графа в ширину. Уметь реализовывать алгоритм с посещением вершины по уровням удаленности от исходной вершины.

Лабораторное задание

Задание 1

1. Сгенерировать (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Вывести матрицу на экран.
2. Для сгенерированного графа осуществить процедуру обхода в ширину, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди использовать класс **queue** из стандартной библиотеки C++.
3. Реализовать процедуру обхода в ширину для графа, представленного списками смежности.

Задание 2

1. Для матричной формы представления графов реализуйте алгоритм обхода в ширину с использованием очереди, построенной на основе структуры данных «список», самостоятельно созданной в лабораторной работе № 3.
2. Оцените время работы двух реализаций алгоритмов обхода в ширину (использующего стандартный класс `queue` и использующего очередь, реализованную самостоятельно) для графов разных порядков.

Описание метода решения задачи

При обходе графа в ширину, мы сначала посетим вершины первого уровня удаленности, и только после того, как закончатся не посещенные вершины на этом уровне, мы перейдем к следующему. Порядок посещения вершин будет храниться в очереди `queue`. Просматривая строку матрицы смежности (или список смежности) для текущей вершины мы помещаем всех её ещё не посещенных соседей в очередь.

Листинг программы

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#include <queue>
#include <vector>
#include <list>
#include <chrono>
#define _CRT_SECURE_NO_WARNINGS
using namespace std;

void BFS(int** G, int* vis, int numG, int cur) {
    std::queue<int> q;
    vis[cur] = 1;
    q.push(cur);
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        printf("%3d", v);
        for (int i = 0; i < numG; i++) {
            if (vis[i] == 0 && G[v][i] == 1) {
                q.push(i);
                vis[i] = 1;
            }
        }
    }
}

void BFS_silent(int** G, int* vis, int numG, int cur) {
    std::queue<int> q;
    vis[cur] = 1;
```

```

    q.push(cur);
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        for (int i = 0; i < numG; i++) {
            if (vis[i] == 0 && G[v][i] == 1) {
                q.push(i);
                vis[i] = 1;
            }
        }
    }
}

```

```

struct AdjListNode {
    int dest;
    AdjListNode* next;
};

```

```

struct AdjList {
    AdjListNode* head;
};

```

```

class Graph {
private:
    int V;
    AdjList* array;

public:
    Graph(int V) {
        this->V = V;
        array = new AdjList[V];
        for (int i = 0; i < V; ++i)
            array[i].head = NULL;
    }
};

```

```
}
```

```
void addEdge(int src, int dest) {  
    AdjListNode* newNode = new AdjListNode;  
    newNode->dest = dest;  
    newNode->next = nullptr;  
  
    if (array[src].head == nullptr) {  
        array[src].head = newNode;  
    }  
    else {  
        AdjListNode* current = array[src].head;  
        while (current->next != nullptr) {  
            current = current->next;  
        }  
        current->next = newNode;  
    }  
  
    newNode = new AdjListNode;  
    newNode->dest = src;  
    newNode->next = nullptr;  
  
    if (array[dest].head == nullptr) {  
        array[dest].head = newNode;  
    }  
    else {  
        AdjListNode* current = array[dest].head;  
        while (current->next != nullptr) {  
            current = current->next;  
        }  
        current->next = newNode;  
    }  
}
```

```

void BFS_adj(int start) {
    int* visited = new int[V];
    for (int i = 0; i < V; i++)
        visited[i] = 0;

    queue<int> q;
    visited[start] = 1;
    q.push(start);

    while (!q.empty()) {
        int v = q.front();
        q.pop();
        printf("%3d", v);

        AdjListNode* pCrawl = array[v].head;
        while (pCrawl != NULL) {
            if (!visited[pCrawl->dest]) {
                visited[pCrawl->dest] = 1;
                q.push(pCrawl->dest);
            }
            pCrawl = pCrawl->next;
        }
    }
    delete[] visited;
}

~Graph() {
    for (int i = 0; i < V; ++i) {
        AdjListNode* current = array[i].head;
        while (current != NULL) {
            AdjListNode* next = current->next;
            delete current;
        }
    }
}

```

```

        current = next;
    }
}
delete[] array;
}
};

```

```

class CustomQueue {
private:
    struct Node {
        int data;
        Node* next;
        Node(int value) : data(value), next(nullptr) {}
    };

    Node* front;
    Node* rear;

public:
    CustomQueue() : front(nullptr), rear(nullptr) {}

    ~CustomQueue() {
        while (!empty()) {
            pop();
        }
    }

    void push(int value) {
        Node* newNode = new Node(value);
        if (rear == nullptr) {
            front = rear = newNode;
        }
        else {

```

```

        rear->next = newNode;
        rear = newNode;
    }
}

```

```

void pop() {
    if (empty()) return;

    Node* temp = front;
    front = front->next;

    if (front == nullptr) {
        rear = nullptr;
    }

    delete temp;
}

```

```

int getFront() {
    if (empty()) {
        return -1;
    }
    return front->data;
}

```

```

bool empty() const {
    return front == nullptr;
}

```

```

size_t size() const {
    size_t count = 0;
    Node* current = front;
    while (current != nullptr) {

```



```

        count++;
        current = current->next;
    }
    return count;
}
};

```

```

void BFS_custom_queue(int** G, int* vis, int numG, int cur) {
    CustomQueue q;
    vis[cur] = 1;
    q.push(cur);
    while (!q.empty()) {
        int v = q.getFront();
        q.pop();
        printf("%3d", v);
        for (int i = 0; i < numG; i++) {
            if (vis[i] == 0 && G[v][i] == 1) {
                q.push(i);
                vis[i] = 1;
            }
        }
    }
}
}

```

```

void BFS_custom_queue_silent(int** G, int* vis, int numG, int cur) {
    CustomQueue q;
    vis[cur] = 1;
    q.push(cur);
    while (!q.empty()) {
        int v = q.getFront();
        q.pop();
        for (int i = 0; i < numG; i++) {
            if (vis[i] == 0 && G[v][i] == 1) {

```

```

        q.push(i);
        vis[i] = 1;
    }
}
}
}

```

```

Graph* createGraphFromMatrix(int** matrix, int n) {
    Graph* graph = new Graph(n);
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (matrix[i][j] == 1) {
                graph->addEdge(i, j);
            }
        }
    }
    return graph;
}

```

```

template<typename Func>
double measureTime(Func func) {
    auto start = chrono::high_resolution_clock::now();
    func();
    auto end = chrono::high_resolution_clock::now();
    chrono::duration<double> duration = end - start;
    return duration.count();
}

```

```

int main() {
    setlocale(LC_ALL, "Russian");
    int** G;
    int* visited;
    int numG, cur;

```

```

srand(time(NULL));

printf("Введите количество вершин графа: ");
scanf_s("%d", &numG);

G = (int**)malloc(numG * sizeof(int*));
for (int i = 0; i < numG; i++) {
    G[i] = (int*)malloc(numG * sizeof(int));
}
visited = (int*)malloc(numG * sizeof(int));

for (int i = 0; i < numG; i++) {
    visited[i] = 0;
    for (int j = i; j < numG; j++) {
        G[i][j] = G[j][i] = (i == j ? 0 : rand() % 10 < 7);
    }
}

printf("Матрица смежности:\n");
for (int i = 0; i < numG; i++) {
    for (int j = 0; j < numG; j++) {
        printf("%3d", G[i][j]);
    }
    printf("\n");
}

printf("Введите стартовую вершину для обхода: ");
scanf_s("%d", &cur);

printf("\n--- Задание 1.2: BFS с матрицей смежности (std::queue) ---\n");
for (int i = 0; i < numG; i++) visited[i] = 0;
BFS(G, visited, numG, cur);
printf("\n");

```

```

printf("\n--- Задание 1.3*: BFS со списками смежности ---\n");
Graph* graph = createGraphFromMatrix(G, numG);
graph->BFS_adj(cur);
printf("\n");
delete graph;

printf("\n--- Задание 2.1: BFS с собственной очередью ---\n");
for (int i = 0; i < numG; i++) visited[i] = 0;
BFS_custom_queue(G, visited, numG, cur);
printf("\n");

printf("\n--- Задание 2.2: Сравнение времени работы ---\n");

int testSizes[] = { 100, 500, 1000 };

for (int t = 0; t < 3; t++) {
    int testSize = testSizes[t];
    printf("\nТестирование для графа размера %d:\n", testSize);

    int** testMatrix = (int**)malloc(testSize * sizeof(int*));
    for (int i = 0; i < testSize; i++) {
        testMatrix[i] = (int*)malloc(testSize * sizeof(int));
        for (int j = 0; j < testSize; j++) {
            if (i == j) testMatrix[i][j] = 0;
            else testMatrix[i][j] = rand() % 2;
        }
    }

    int* testVisited = (int*)malloc(testSize * sizeof(int));

    for (int i = 0; i < testSize; i++) testVisited[i] = 0;
    double timeStd = measureTime([&]() {

```

```

        BFS_silent(testMatrix, testVisited, testSize, 0);
    });

    for (int i = 0; i < testSize; i++) testVisited[i] = 0;
    double timeCustom = measureTime([&]() {
        BFS_custom_queue_silent(testMatrix, testVisited, testSize, 0);
    });

    printf("Стандартная очередь: %.6f сек\n", timeStd);
    printf("Собственная очередь: %.6f сек\n", timeCustom);
    printf("Разница: %.6f сек\n", timeCustom - timeStd);

    for (int i = 0; i < testSize; i++) {
        free(testMatrix[i]);
    }
    free(testMatrix);
    free(testVisited);
}

for (int i = 0; i < numG; i++) {
    free(G[i]);
}
free(G);
free(visited);

return 0;
}

```

Результат работы программы:

```
Консоль отладки Microsoft Visual Studio
Введите количество вершин графа: 5
Матрица смежности:
  0 1 1 1 0
  1 0 0 1 0
  1 0 0 1 1
  1 1 1 0 1
  0 0 1 1 0
Введите стартовую вершину для обхода: 3

--- Задание 1.2: BFS с матрицей смежности (std::queue) ---
  3 0 1 2 4

--- Задание 1.3*: BFS со списками смежности ---
  3 0 1 2 4

--- Задание 2.1: BFS с собственной очередью ---
  3 0 1 2 4

--- Задание 2.2: Сравнение времени работы ---

Тестирование для графа размера 100:
Стандартная очередь: 0,000154 сек
Собственная очередь: 0,000098 сек
Разница: -0,000056 сек

Тестирование для графа размера 500:
Стандартная очередь: 0,000946 сек
Собственная очередь: 0,000475 сек
Разница: -0,000471 сек

Тестирование для графа размера 1000:
Стандартная очередь: 0,000940 сек
Собственная очередь: 0,001338 сек
Разница: 0,000397 сек

C:\Users\ardor\source\repos\andr8\x64\Debug\andr8.exe (процесс 15692) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно: _
```

Вывод: научился реализовывать алгоритм для обхода графа в ширину. Научился реализовывать алгоритм с посещением вершины по уровням удаленности от исходной вершины.